

Semgrep 规则编写与漏洞分析实验报告

学号：25140916 姓名：王玲玉

Successfully installed anyio-4.13.0 boltons-21.0.0 brace-ex-2.6 click-8.1.8 click-option-group-0.5.9 exceptiongroup-1.2.2 face-26.0.0 glom-25.12.0 googleapis-common-protos-1.74.0 httpx-0.28.1 httpx-sse-0.4.3 isonenschema-4.25.1 mcp-1.23.3 opentelemetry-api-1.37.0 opentelemetry-exporter-otlp-proto-common-1.37.0 opentelemetry-exporter-otlp-proto-http-1.37.0 opentelemetry-instrumentation-0.58b0 opentelemetry-instrumentation-threading-0.58b0 opentelemetry-proto-1.37.0 opentelemetry-sdk-1.37.0 opentelemetry-semantic-conventions-0.58b0 opentelemetry-util-http-0.58b0 pae-wee-3.19.0 protobuf-6.33.0 pydantic-2.13.3 pydantic-core-2.46.3 pyjwt-2.12.1 python-multipart-0.8.26 ruamel.yaml-0.19.1 ruamel.yaml.clib-0.2.14 semantic-version-2.10.0 semgrep-1.161.0 sse-starlette-3.3.4 starlette-1.0.0 typing-extensions-4.15.0 typing-inspection-0.4.2 uvicorn-0.46.0 wcmatch-8.5.2

(base) wanglingyudeMacBook-Pro:git-lab-25140916 wly\$ ls

code-repo notes.md readme.md

(base) wanglingyudeMacBook-Pro:git-lab-25140916 wly\$

(base) wanglingyudeMacBook-Pro:git-lab-25140916 wly\$ semgrep -f /Users/wly/Documents/open_source_study/rule1.txt code-repo/

```

Semgrep CLI

Loading rules...                               Scanning 13 files (only git-tracked) with 1 Code rule:

CODE RULES
Scanning 3 files.

SUPPLY CHAIN RULES

No rules to run.

PROGRESS
100% 0:00:00
```

```

3 Code Findings

code-repo/db.py
>>> Users.wly.Documents.open_source_study.intranel.python.sql-
injection.tainted-sql-execute
(( Blocking ))
User-controlled input flows into a SQL execution sink. Do
not build SQL statements with string formatting, f-strings,
concatenation, or .format(). Use parameterized queries such
as cur.execute("SELECT ... WHERE username = ?",
(username,)).

40: cur.execute(query)
51: cur.execute(query)
64: cur.execute(query)
```

```

Scan Summary

✓ Scan completed successfully.
• Findings: 3 (3 blocking)
• Rules run: 1
• Targets scanned: 3
• Parsed lines: ~100.0%
• Scan was limited to files tracked by git
• For a detailed list of skipped files and lines, run semgrep with the --verbose flag
Ran 1 rule on 3 files: 3 findings.
(base) wanglingyudeMacBook-Pro:git-lab-25140916 wly$
```

```

User-controlled input flows into a SQL execution sink. Do
not build SQL statements with string formatting, f-strings,
concatenation, or .format(). Use parameterized queries such
as cur.execute("SELECT ... WHERE username = ?",
(username,)).

40: cur.execute(query)
51: cur.execute(query)
64: cur.execute(query)
```

```

Scan Summary

✓ Scan completed successfully.
• Findings: 3 (3 blocking)
• Rules run: 1
• Targets scanned: 3
• Parsed lines: ~100.0%
• Scan was limited to files tracked by git
• For a detailed list of skipped files and lines, run semgrep with the --verbose flag
Ran 1 rule on 3 files: 3 findings.
(base) wanglingyudeMacBook-Pro:git-lab-25140916 wly$ ls
code-repo  notes.md  readme.md
(base) wanglingyudeMacBook-Pro:git-lab-25140916 wly$
(base) wanglingyudeMacBook-Pro:git-lab-25140916 wly$ cd code-repo/
(base) wanglingyudeMacBook-Pro:code-repo wly$ ls
app.py      requirements.txt  SQL 注入漏洞分析.docx  utils.py
db.py       rule1.txt        templates
(base) wanglingyudeMacBook-Pro:code-repo wly$ git status
On branch main
Your branch is up to date with 'origin/main'.
```

```

Untracked files:
(use "git add <file>..." to include in what will be committed)
../.DS_Store
"SQL [346\263\250\345\205\245\346\274\217\346\264\236\345\210\206\346\236\220.docx"
rule1.txt

nothing added to commit but untracked files present (use "git add" to track)
(base) wanglingyudeMacBook-Pro:code-repo wly$ git add .
(base) wanglingyudeMacBook-Pro:code-repo wly$ git commit -m"add"
(main cded9a0) add
2 files changed, 69 insertions(+)
create mode 100644 "code-repo/SQL [346\263\250\345\205\245\346\274\217\346\264\236\345\210\206\346\236\220.docx"
create mode 100644 code-repo/rule1.txt
(base) wanglingyudeMacBook-Pro:code-repo wly$ git push origin master
error: src refspec master does not match any
error: failed to push some refs to 'git@github.com:wanglingyu11/git-lab-25140916.git'
(base) wanglingyudeMacBook-Pro:code-repo wly$ git push origin main
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 8 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (5/5), 873.88 KiB | 2.93 MiB/s, done.
Total 5 (delta 1), reused 0 (delta 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To github.com:wanglingyu11/git-lab-25140916.git
3b48e62..cded9a0 main -> main
(base) wanglingyudeMacBook-Pro:code-repo wly$
```

规则内容

rules:

- id: intranel.python.sql-injection.tainted-sql-execute
- languages: [python]
- severity: ERROR

mode: taint

message: > -

User-controlled input flows into a SQL execution sink. Do not build SQL statements with string formatting, f-strings, concatenation, or .format().

Use parameterized queries such as cur.execute("SELECT ... WHERE username = ?", (username,)).

metadata:

category: security

confidence: HIGH

cwe: "CWE-89: Improper Neutralization of Special Elements used in an SQL Command"

owasp: "A03:2021 Injection"

technology: [python, sqlite, flask]

source: "Flask request input or db.py function parameters"

sink: "cursor.execute / connection.execute / executescript / executemany"

attack_flow: "user input -> SQL string formatting -> database execution"

pattern-sources:

1. Flask HTTP input sources.

- pattern: request.args.get(...)
- pattern: request.args[...]
- pattern: request.form.get(...)
- pattern: request.form[...]
- pattern: request.values.get(...)
- pattern: request.values[...]
- pattern: request.get_json(...)
- pattern: request.json
- pattern: request.cookies.get(...)
- pattern: request.headers.get(...)

2. Explicitly model db.py parameters as tainted sources.

This catches cases where request data is passed into helper functions

such as find_user(username), find_user_by_email(email), create_user(...).

- patterns:

- pattern-inside: |
def find_user(\$USERNAME):
...
- pattern: \$USERNAME

- patterns:

- pattern-inside: |
def find_user_by_email(\$EMAIL):
...
- pattern: \$EMAIL

- patterns:
 - pattern-inside: |


```
def create_user($USERNAME, $PASSWORD, $EMAIL):
    ...
```
 - pattern-either:
 - pattern: \$USERNAME
 - pattern: \$PASSWORD
 - pattern: \$EMAIL

pattern-sinks:

sqlite cursor / connection execute sinks.
 # focus-metavariable ensures the taint must flow into the SQL argument,
 # not merely into the parameter tuple of a safe parameterized query.

- patterns:
 - pattern-either:
 - pattern: \$CUR.execute(\$SQL, ...)
 - pattern: \$CONN.execute(\$SQL, ...)
 - pattern: \$CUR.executemany(\$SQL, ...)
 - pattern: \$CONN.executemany(\$SQL, ...)
 - pattern: \$CUR.executescript(\$SQL)
 - pattern: \$CONN.executescript(\$SQL)
 - focus-metavariable: \$SQL

SQL 注入漏洞分析

在本项目中，db.py 中存在多处 SQL 注入风险，主要涉及用户查询、邮箱查询和用户创建等数据库操作逻辑。该类漏洞的本质是：应用程序将来自外部输入或业务参数的用户可控数据，直接通过字符串拼接、格式化或模板插值的方式嵌入 SQL 语句中，而不是使用数据库驱动提供的参数化查询机制。攻击者可以构造特殊输入改变 SQL 语句原有语义，从而绕过认证、越权查询、篡改数据，甚至在特定数据库配置下执行破坏性操作。该问题在已有安全分析中被归类为 Critical 级漏洞。

1. 漏洞成因

1.1 find_user 函数中的 SQL 拼接

find_user 函数用于根据用户名查询用户信息。代码中使用 % 字符串格式化方式将 username 直接拼接进 SQL 查询语句：

```
query = "SELECT * FROM users WHERE username = '%s'" % username
```

这里的 username 是用户可控输入，属于漏洞分析中的 **源 (source)**。该变量未经任何参数化处理，直接进入 SQL 字符串，并最终被数据库执行接口执行，形成 SQL 注入风险。

如果攻击者输入的用户名中包含 SQL 控制字符，例如单引号、注释符或布尔表达式，就可能改变原本的查询条件。原本系统预期执行的是：

```
SELECT * FROM users WHERE username = '<username>'
```

但在恶意输入参与拼接后，查询条件可能被扩展或绕过，从而返回非预期用户数据。

1.2 find_user_by_email 函数中的 f-string 拼接

find_user_by_email 函数中使用 f-string 构造 SQL:

```
query = f"SELECT * FROM users WHERE email = '{email}'"
```

f-string 只是 Python 层面的字符串插值, 不具备任何 SQL 安全处理能力。email 一旦包含特殊 SQL 片段, 仍会被原样写入查询语句。因此, 该写法与普通字符串拼接在安全风险上没有本质区别。

此处风险在于, 邮箱字段通常来自登录、注册、找回密码或用户搜索等外部输入场景。如果攻击者可以控制 email 参数, 就可能构造异常查询条件, 绕过正常的邮箱匹配逻辑, 读取其他用户数据或辅助进行账户枚举。

1.3 create_user 函数中的 .format() 拼接

create_user 函数中使用 .format() 拼接插入语句:

```
query = "INSERT INTO users (username, pw_hash, email) VALUES ('{}', '{}', '{}')".format(...)
```

该问题不仅影响查询语句, 也影响写入语句。攻击者如果能控制 username、email 或其他字段内容, 就可能破坏原有 INSERT 语句结构, 造成非预期字段写入、语句截断、插入异常, 甚至在数据库驱动和配置允许多语句执行的情况下触发更高风险操作。

因此, SQL 注入并不只存在于 SELECT 查询中, 所有通过字符串拼接构造的 INSERT、UPDATE、DELETE 等语句都应纳入审计范围。

2. 攻击过程

该漏洞的攻击过程可以概括为以下几个阶段。

首先, 攻击者通过登录、注册、用户搜索或邮箱查询等功能入口提交构造后的输入。该输入在业务语义上可能看似是用户名或邮箱, 但其中包含 SQL 特殊字符、逻辑表达式或注释符。其次, 应用程序未将该输入作为独立参数交给数据库驱动处理, 而是通过 %、f-string 或 .format() 将其直接拼接进 SQL 语句。此时, 用户输入已经从“普通数据”变成了 SQL 语句的一部分。

然后, 拼接后的 SQL 被传入数据库执行接口, 例如 cursor.execute(query)。数据库解析该语句时, 并不知道哪些内容原本属于用户输入, 哪些内容属于开发者预期的 SQL 结构, 因此会按照完整 SQL 语法进行解释执行。

最终, 攻击者可能改变查询条件、绕过认证逻辑、读取非授权数据、写入异常数据, 或破坏数据库完整性。

3. 源、传播路径与汇

从静态分析角度看, 该漏洞可以建模为典型的污点传播问题。

类型	示例	安全含义
Source	username、email、注册字段、搜索字段	用户可控输入
Propagation	% 格式化、f-string、.format()、字符串拼接	用户输入被拼入 SQL 字符串
Sink	cur.execute(query)、cursor.execute(query)	SQL 语句被数据库执行

该漏洞的关键判定条件不是“是否执行了 SQL”, 而是是否同时满足以下条件:

1. SQL 语句中包含用户可控输入;
2. 用户输入通过字符串拼接、格式化或模板插值进入 SQL;
3. 数据库执行接口接收的是拼接后的完整 SQL 字符串;
4. 未使用参数化查询或预编译语句隔离 SQL 结构与用户数据。

4. 影响分析

SQL 注入属于严重级别漏洞, 其影响范围取决于数据库权限、业务逻辑和部署环境。常见影响包括:

影响类型	说明
认证绕过	攻击者可能绕过用户名、邮箱或密码校验逻辑
数据泄露	可读取用户表、邮箱、密码哈希、权限字段等敏感数据
数据篡改	可修改账户信息、权限状态或业务数据
数据破坏	在高权限或多语句执行条件下，可能删除表或破坏数据库结构
业务逻辑绕过	可能绕过搜索、授权、注册校验等业务限制
后续攻击支撑	泄露的密码哈希、邮箱和权限信息可用于撞库、钓鱼或横向移动

在当前项目中，相关 SQL 操作涉及用户查询和用户创建逻辑，因此风险不仅限于数据读取，还可能影响账户体系完整性和身份认证安全。

5. 修复建议

修复该类漏洞的核心原则是：**永远不要通过字符串拼接构造包含用户输入的 SQL 语句**。应使用数据库驱动提供的参数化查询机制，将 SQL 结构与用户数据分离。

对于 SQLite，应使用 `?` 占位符：

```
cur.execute(
    "SELECT * FROM users WHERE username = ?",
    (username,)
)
```

邮箱查询应修改为：

```
cur.execute(
    "SELECT * FROM users WHERE email = ?",
    (email,)
)
```

用户创建逻辑应修改为：

```
cur.execute(
    "INSERT INTO users (username, pw_hash, email) VALUES (?, ?, ?)",
    (username, pw_hash, email)
)
```

这种写法中，SQL 语句结构由开发者固定，用户输入只作为参数值传递给数据库驱动。即使输入中包含单引号、注释符或 SQL 关键字，也只会当作普通字符串处理，不会改变 SQL 语义。

6. 安全加固建议

除参数化查询外，还建议结合以下措施进行整体加固：

加固方向	说明
输入校验	对用户名、邮箱等字段设置格式、长度和字符集约束
最小权限	数据库账号仅授予应用所需的最小权限
错误信息控制	避免将 SQL 报错、堆栈信息直接返回给前端
统一数据访问层	封装数据库访问函数，禁止业务代码自行拼接 SQL
安全测试	在 CI 中加入 Semgrep、Bandit 或自定义 SQL 注入规则
日志审计	记录异常查询、频繁失败登录和异常注册行为
密码安全	与 SQL 注入修复同步检查密码哈希、账户锁定和认证流程

7. 结论

该项目中的 SQL 注入问题属于典型的高危输入处理缺陷。find_user、find_user_by_email 和 create_user 分别通过 %、f-string 和 .format() 将用户可控数据直接拼入 SQL 语句，导致数据库执行阶段无法区分“代码”和“数据”。一旦攻击者能够控制相关输入字段，就可能改变 SQL 查询或写入逻辑，造成认证绕过、敏感信息泄露、数据篡改或数据库破坏。

建议将相关数据库操作全部改造为参数化查询，并将“禁止拼接 SQL”作为项目级安全编码规范纳入后续审计和自动化检测流程。